créer un fichier et à y enregistrer les différentes données.

J'ai ainsi pu identifier les périphériques et les bibliothèques nécessaires. Voilà à quoi ressemble le début du programme :

Code source de interruption.c :

```c
#include "grove_rtc_ds1307.h"
#include "grove_gps_air530z.h"
#include "grove_light.h"
#include "sd_logger.h"

#include <stdio.h>
#include <string.h>

/*
 * Les périphériques suivants sont initialisés par les fonctions
 * MX_..._Init() du projet :
 *
 * - I2C1 : horloge RTC ;
 * - USART1 : GPS ;
 * - USART2 : terminal série par le ST-LINK ;
 * - ADC1 : capteur de luminosité ;
 * - SPI1 : carte SD.
 */

extern I2C_HandleTypeDef hi2c1;
extern UART_HandleTypeDef huart1;
extern UART_HandleTypeDef huart2;
extern ADC_HandleTypeDef hadc1;
extern SPI_HandleTypeDef hspi1;

int main(void)
{
 GroveRTC_DateTime_t date_time;

 uint16_t light_value = 0;

 char gps_data[100];
 char data_line[256];

 HAL_Init();
 SystemClock_Config();

 MX_GPIO_Init();
 MX_ADC1_Init();
 MX_I2C1_Init();
 MX_SPI1_Init();
 MX_USART1_UART_Init();
 MX_USART2_UART_Init();

 /*
 * Initialisation des modules du projet.
 */
 GroveRTC_Init(&hi2c1);
 GroveGPS_Init(&huart1);
 GroveLight_Init(&hadc1);

 printf("Initialisation de la carte SD...\r\n");

 if (SDLogger_Init(&hspi1, SD_CS_GPIO_Port, SD_CS_Pin) != HAL_OK)
 {
 printf("Carte SD absente ou impossible à initialiser.\r\n");

 /*
 * Le programme reste bloqué si la carte SD ne peut pas être utilisée.
 */
 while (1)
 {
```

```
/*
* Le programme reste bloqué si la carte SD ne peut pas être utilisée.
*/
while (1)
{
}
}

printf("Carte SD initialisée.\r\n");

while (1)
{
/*
* Lecture de la date et de l'heure depuis l'horloge RTC.
*/
if (GroveRTC_GetDateTime(&date_time) != HAL_OK)
{
printf("Erreur de lecture de l'horloge RTC.\r\n");
}

/*
* Lecture de la valeur brute du capteur de luminosité.
*/
if (GroveLight_ReadRaw(&light_value) != HAL_OK)
{
printf("Erreur de lecture du capteur de luminosité.\r\n");
light_value = 0;
}

/*
* Lecture des données envoyées par le GPS.
*
* Le programme lit les trames reçues jusqu'à trouver une trame GGA,
* qui contient notamment la position géographique.
*/
gps_data[0] = '\0';

do
{
if (GroveGPS_Read(gps_data,
sizeof(gps_data),
1000U) != HAL_OK)
{
strcpy(gps_data, "GPS indisponible");
break;
}
}
while (strncmp(gps_data, "$GPGGA", 6) != 0);

/*
* Création d'une ligne regroupant toutes les données.
*
* Exemple :
* 14:35:12 01/01/2026 ; 1850 ; $GPGGA,...
*/
snprintf(data_line,
sizeof(data_line),
"%02u:%02u:%02u %02u/%02u/%04u ; %u ; %s",
date_time.hour,
date_time.minute,
date_time.second,
date_time.day,
date_time.month,
date_time.year,
light_value,
gps_data);
```

```
*/
snprintf(data_line,
sizeof(data_line),
"%02u:%02u:%02u %02u/%02u/%04u ; %u ; %s",
date_time.hour,
date_time.minute,
date_time.second,
date_time.day,
date_time.month,
date_time.year,
light_value,
gps_data);

/*
* Enregistrement de la ligne dans le fichier de mesures.
*/
if (SDLogger_WriteLine("datalog.txt", data_line) != HAL_OK)
{
printf("Erreur lors de l'écriture dans datalog.txt.\r\n");
}
else
{
/*
* Affichage de la même ligne dans le terminal série.
*/
printf("%s\r\n", data_line);
}

HAL_Delay(1000);
}
}
```

Le reste de l'équipe regarde le code, incrédule...

> Ça y est, j'ai un super programme ! J'ai réussi à récupérer les informations provenant des principaux composants de la station :
>
> Le GPS m'a tout de même posé quelques difficultés. Il communique à l'aide d'une liaison série UART, mais nous utilisons également une liaison série pour configurer le système et afficher des informations dans le terminal.
>
> Lors de mes premiers essais, les données envoyées par le GPS se mélangeaient avec les messages destinés à l'utilisateur. L'interface devenait difficile à comprendre : les trames de géolocalisation apparaissaient au milieu des paramètres et des messages de configuration.
>
> Il n'est donc pas envisageable d'utiliser la même interface série pour le GPS et pour la communication avec l'utilisateur.
>
> Heureusement, le microcontrôleur STM32L476RG possède plusieurs périphériques UART. J'ai donc utilisé :
>
> - USART1 pour communiquer avec le GPS ;
> - USART2 pour communiquer avec le terminal par l'intermédiaire du port série virtuel du ST-LINK.
>
> Pour le capteur de luminosité, connecté à une entrée analogique, je me contente pour le moment d'enregistrer la valeur mesurée, sans effectuer de traitement particulier.
>
> La carte SD communique avec le microcontrôleur par l'intermédiaire du bus SPI. Je parviens désormais à créer un fichier et à y enregistrer les différentes données.
>
> J'ai ainsi pu identifier les périphériques et les bibliothèques nécessaires. Voilà à quoi ressemble le programme :

Code source de main.c :

```
/*
* LED utilisateur LD2 de la carte NUCLEO-L476RG :
* PA5, active à l'état haut.
*/
#define LED_GPIO_PORT GPIOA
#define LED_PIN GPIO_PIN_5

/*
* Bouton utilisateur B1 de la carte NUCLEO-L476RG :
* PC13, actif à l'état bas.
*/
#define BUTTON_GPIO_PORT GPIOC
```

- UART2 pour communiquer avec le terminal par l'intermédiaire du port série virtuel du ST-LINK.

Pour le capteur de luminosité, connecté à une entrée analogique, je me contente pour le moment d'enregistrer la valeur mesurée, sans effectuer de traitement particulier.

La carte SD communique avec le microcontrôleur par l'intermédiaire du bus SPI. Je parviens désormais à créer un fichier et à y enregistrer les différentes données.

J'ai ainsi pu identifier les périphériques et les bibliothèques nécessaires. Voilà à quoi ressemble le programme :

Code source de main.c :

```c
/*
 * LED utilisateur LD2 de la carte NUCLEO-L476RG :
 * PA5, active à l'état haut.
 */
#define LED_GPIO_PORT GPIOA
#define LED_PIN GPIO_PIN_5

/*
 * Bouton utilisateur B1 de la carte NUCLEO-L476RG :
 * PC13, actif à l'état bas.
 */
#define BUTTON_GPIO_PORT GPIOC
#define BUTTON_PIN GPIO_PIN_13
#define BUTTON_PRESSED GPIO_PIN_RESET

static void GPIO_Init(void);
static void acquisition(void);

int main(void)
{
 GPIO_PinState button_state;

 HAL_Init();
 SystemClock_Config();

 GPIO_Init();

 while (1)
 {
 /*
 * Lecture de l'état du bouton par scrutation.
 *
 * Cette lecture n'est réalisée qu'une seule fois avant le lancement
 * de la fonction acquisition().
 */
 button_state = HAL_GPIO_ReadPin(BUTTON_GPIO_PORT, BUTTON_PIN);

 if (button_state == BUTTON_PRESSED)
 {
 HAL_GPIO_WritePin(LED_GPIO_PORT,
 LED_PIN,
 GPIO_PIN_SET);
 }
 else
 {
 HAL_GPIO_WritePin(LED_GPIO_PORT,
 LED_PIN,
 GPIO_PIN_RESET);
 }

 /*
 * Simulation d'une acquisition longue.
 *
 * Pendant cette fonction, l'état du bouton n'est plus consulté.
 */
 acquisition();
 }
}
```

```
37
38 if (button_state == BUTTON_PRESSED)
39 {
40 HAL_GPIO_WritePin(LED_GPIO_PORT,
41 LED_PIN,
42 GPIO_PIN_SET);
43 }
44 else
45 {
46 HAL_GPIO_WritePin(LED_GPIO_PORT,
47 LED_PIN,
48 GPIO_PIN_RESET);
49 }
50
51 /*
52 * Simulation d'une acquisition longue.
53 *
54 * Pendant cette fonction, l'état du bouton n'est plus consulté.
55 */
56 acquisition();
57 }
58}
59
60static void acquisition(void)
61{
62 uint32_t acquisition_duration;
63
64 /*
65 * Simulation d'une durée d'acquisition comprise entre
66 * trois et presque sept secondes.
67 */
68 acquisition_duration = 3000U + (HAL_GetTick() % 4000U);
69
70 /*
71 * HAL_Delay() bloque la poursuite du programme principal.
72 */
73 HAL_Delay(acquisition_duration);
74}
75
76static void GPIO_Init(void)
77{
78 GPIO_InitTypeDef gpio_init = {0};
79
80 /*
81 * Activation des horloges des ports GPIOA et GPIOC.
82 */
83 __HAL_RCC_GPIOA_CLK_ENABLE();
84 __HAL_RCC_GPIOC_CLK_ENABLE();
85
86 /*
87 * Configuration de la LED LD2 en sortie.
88 */
89 gpio_init.Pin = LED_PIN;
90 gpio_init.Mode = GPIO_MODE_OUTPUT_PP;
91 gpio_init.Pull = GPIO_NOPULL;
92 gpio_init.Speed = GPIO_SPEED_FREQ_LOW;
93
94 HAL_GPIO_Init(LED_GPIO_PORT, &gpio_init);
95
96 /*
97 * Configuration du bouton B1 en entrée simple.
98 *
99 * Aucune interruption n'est encore configurée.
100 */
101 gpio_init.Pin = BUTTON_PIN;
102 gpio_init.Mode = GPIO_MODE_INPUT;
```

```
/*
* Configuration de la LED LD2 en sortie.
*/
gpio_init.Pin = LED_PIN;
gpio_init.Mode = GPIO_MODE_OUTPUT_PP;
gpio_init.Pull = GPIO_NOPULL;
gpio_init.Speed = GPIO_SPEED_FREQ_LOW;

HAL_GPIO_Init(LED_GPIO_PORT, &gpio_init);

/*
* Configuration du bouton B1 en entrée simple.
*
* Aucune interruption n'est encore configurée.
*/
gpio_init.Pin = BUTTON_PIN;
gpio_init.Mode = GPIO_MODE_INPUT;
gpio_init.Pull = GPIO_NOPULL;

HAL_GPIO_Init(BUTTON_GPIO_PORT, &gpio_init);

/*
* La LED est éteinte au démarrage.
*/
HAL_GPIO_WritePin(LED_GPIO_PORT,
LED_PIN,
GPIO_PIN_RESET);
}
```

Intrigué par ce problème, un autre membre de l'équipe effectue ses propres essais.

Lorsqu'il appuie sur le bouton, il constate que celui-ci fonctionne correctement. Pourtant, la LED présente un comportement surprenant : son allumage ou son extinction ne sont pas synchronisés avec l'appui.

Certains appuis ne sont pas détectés du tout, tandis que d'autres ne sont pris en compte qu'après plusieurs secondes.

Ressources pour les étudiants

Principes d'interruption

Interruptions Microcontrôleurs

Explications plus générales du site de référence : Embedded.com

Explications EXTI/NVIC STM32

CORBEILLE INTERRUPTIONS

PRÉREQUIS

Liste des prérequis

Programmation en C

SUJET

Ressources pour les étudiants

Corbeille interruptions